

포트폴리오

지원자 서지민

CONTACT

 jn2xxcom@gmail.com

 010 8549 5412

“백엔드 개발부터 인프라 설계·운영까지, 서비스 전 과정을 직접 구축해온 엔지니어입니다.”

클라우드 인프라	AWS	SAA 자격 보유, 4개 프로젝트 실 운영
	Terraform(IaC)	AWS 인프라 전체 코드화
	Kubernetes	항해99 데브코스 수료, ArgoCD 챌린저 참여
CI/CD	Github Actions	ECR 연동 자동 배포 파이프라인 구축
	Bash 스크립트	배포 자동화 스크립트 작성
모니터링	CloudWatch	로그·메트릭 실시간 수집 및 Observability 구축
개발	Spring Boot · Django	3개 프로젝트 백엔드 API 설계 및 구현
DB	MySQL · PostgreSQL	쿼리 최적화 경험, SQLD 자격 보유
기타	Docker · Linux · Git	컨테이너화, 서버 운영, 브랜치 전략 적용

소개



Github
@SeoJimin1234



LinkedIn
@jimin-seo-2xx/



Blog
<https://seojimin1234.github.io/>

EDUCATION

- 2021.03 이화여자대학교 입학
(주: 컴퓨터공학, 부: 사이버보안)
- 2026.08 이화여자대학교 학사 졸업 예정

EXPERIENCE

- 2026.03 - 현재 AWS Cloud Club
- 2026.03 - 현재 연합개발동아리 IT's Time(개발자)
- 2025.09 - 2026.02 Frankfurt UAS 파견 교환학생
- 2025.04 - 2025.05 항해 99 데브코스 쿠버네티스
- 2024.09 - 2025.02 대학생연합개발동아리 UMC(개발자)
- 2024.07 - 2024.07 오픈SW컨트리뷰션아카데미 ArgoCD(챌린저)
- 2024.02 - 2025.08 EWHA-CHAIN(부학회장)
- 2023.09 - 2024.02 이화여대 중앙컴퓨터동아리
- 2023.03 - 2024.03 지속가능발전학회 SDP(테크팀원, 홍보팀장)
- 2021.09 - 2024.12 문화교류연합동아리 MEDDY(홍보팀장)

SKILLS

- High SpringBoot, Django, SQL, Docker
- Middle AWS, Terraform, Linux, Bash

AWARD

- 2024.09 WEB2X ConnecTHON 대상
- 2024.11 이화여대 SW 창업경진대회 장려상
- 2025.05 SW중심대학 창업경진대회 장려상

LICENSE

- 정보처리산업기사 (2024.09 취득)
- AWS SAA (2025.02 취득)
- SQL 개발자 (2025.04 취득)
- 네트워크관리사 2급 (2025.07 취득)
- OPIc 영어 IH (2026.03 취득)

프로젝트 목록

 Re;PLAN Page 04~08 실패 원인을 분석하고, 할 일을 재설계하는 To-Do List 서비스	 플랫폼 Mobile App	 담당 역할 백엔드&인프라 엔지니어 (BE 2명, FE 2명)	 개발 기간 2026.03 - 진행중
 TokenUs Page 09~15 영상을 디지털 자산으로 변환하는 NFT 기반 영상 공유 플랫폼	 플랫폼 Web	 담당 역할 백엔드&인프라 엔지니어 (BE 1명, FE 1명)	 개발 기간 2024.09 - 2025.11
 SASM Page 16~21 지속 가능한 장소를 큐레이팅하고 커뮤니티를 제공하는 서비스	 플랫폼 Mobile App, Web	 담당 역할 백엔드 엔지니어 (BE 4명, FE 4명)	 개발 참여 기간 2023.03 - 2024.02

Re;PLAN

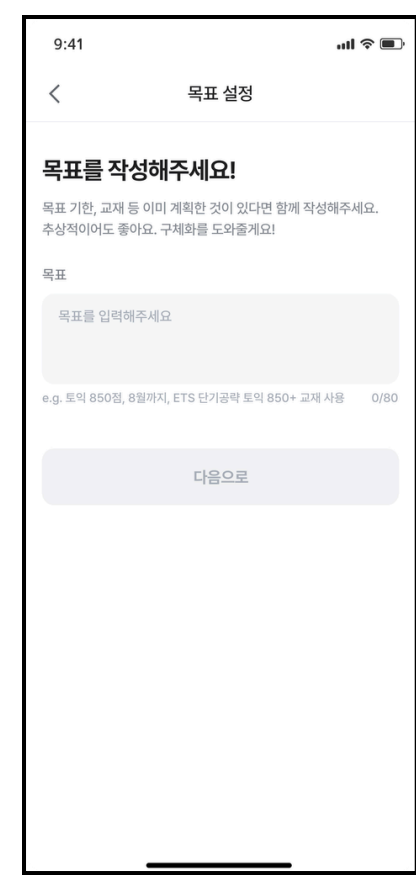


개요

생성형 AI를 활용하여 실패 원인을 분석하고, 이를 바탕으로 할 일을 실행 가능한 계획으로 재설계해주는 To-Do List 서비스

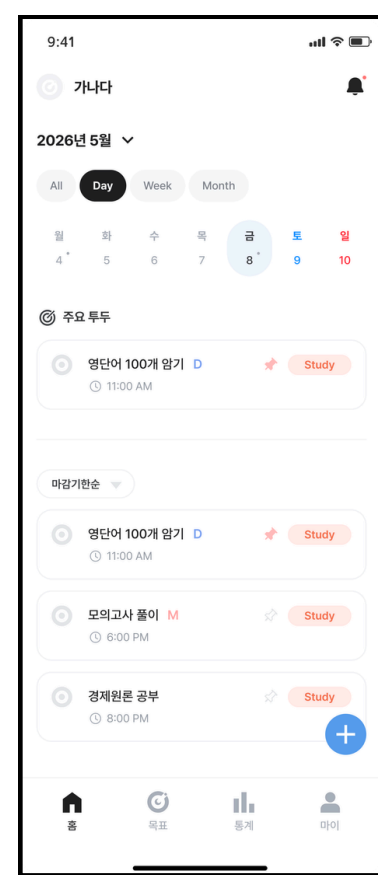
핵심 기능

온보딩



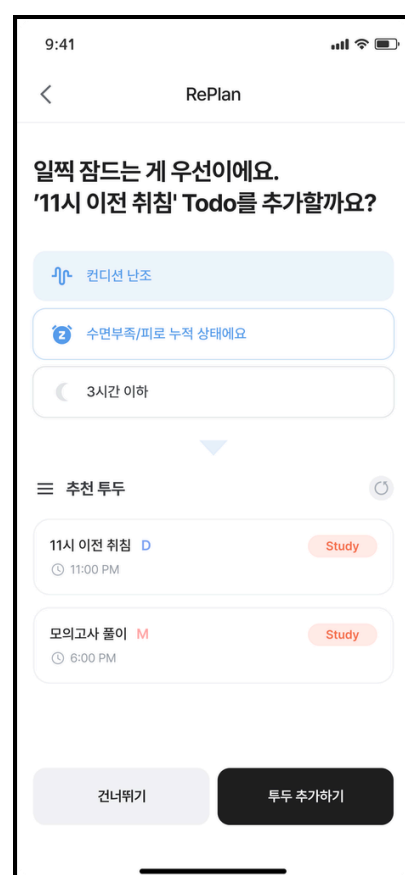
사용자의 목표를 위한
ToDoList 자동 생성

홈



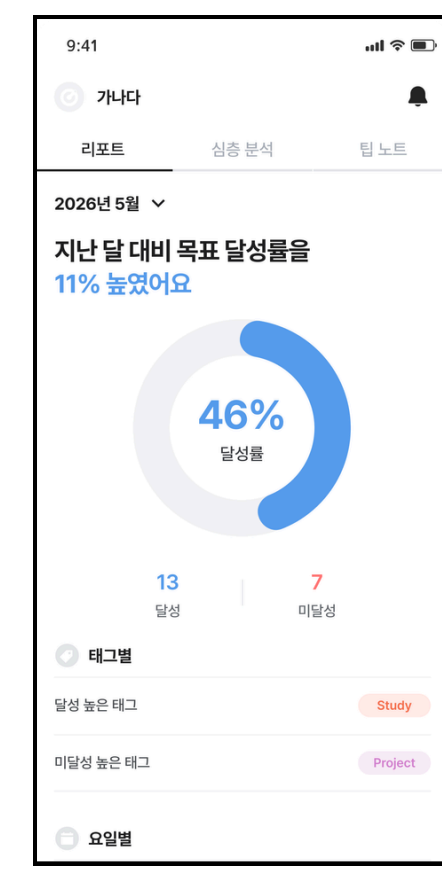
ToDoList 관리

리플랜



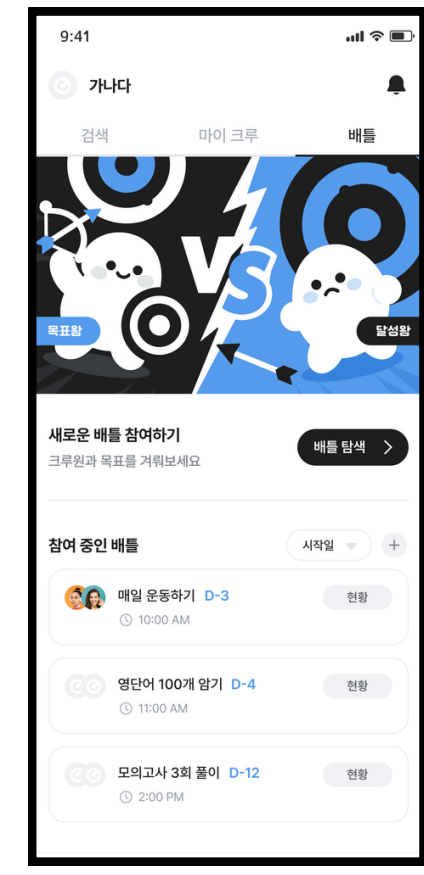
AI기반 실패원인 분석
및 계획 재설계

통계 및 피드백



달성률 통계 및
AI 피드백

크루



다양한 사람들과 함께
도전할 수 있는 커뮤니티

기술스택



Terraform 코드 기반의 인프라 관리(IaC)를 통해 수동 설정 오류를 방지하고, 인프라의 형상 관리 및 재사용성을 극대화하기 위해 채택



SSM

API 키, DB 비밀번호 등 민감한 설정 정보를 소스 코드와 완전히 분리하여 중앙 집중식 보안 관리를 구현하기 위해 사용



CloudWatch

시스템 리소스와 애플리케이션 로그를 실시간으로 수집·시각화하여, 장애 발생 시 신속한 원인 파악 및 대응이 가능한 운영 환경 조성



Redis

데이터베이스의 부하를 분산하고 서비스 응답 성능을 향상시키기 위한 캐싱 레이어 및 분산 환경에서의 세션 관리자로 활용



Nginx

리버스 프록시 설정을 통해 내부 서버 구조를 은닉하고, SSL 인증서를 적용하여 안전한 HTTPS 통신 환경을 제공하기 위해 활용



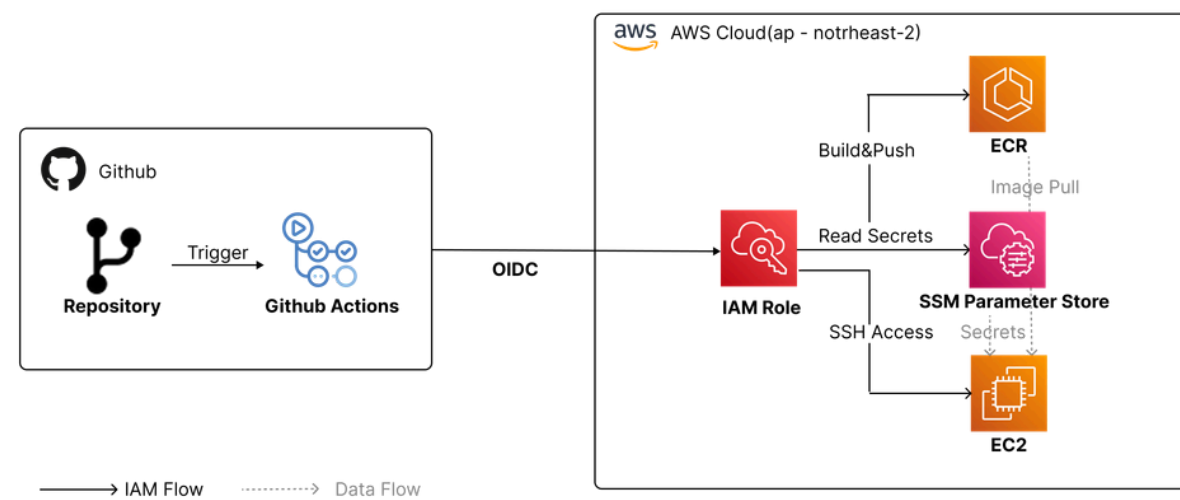
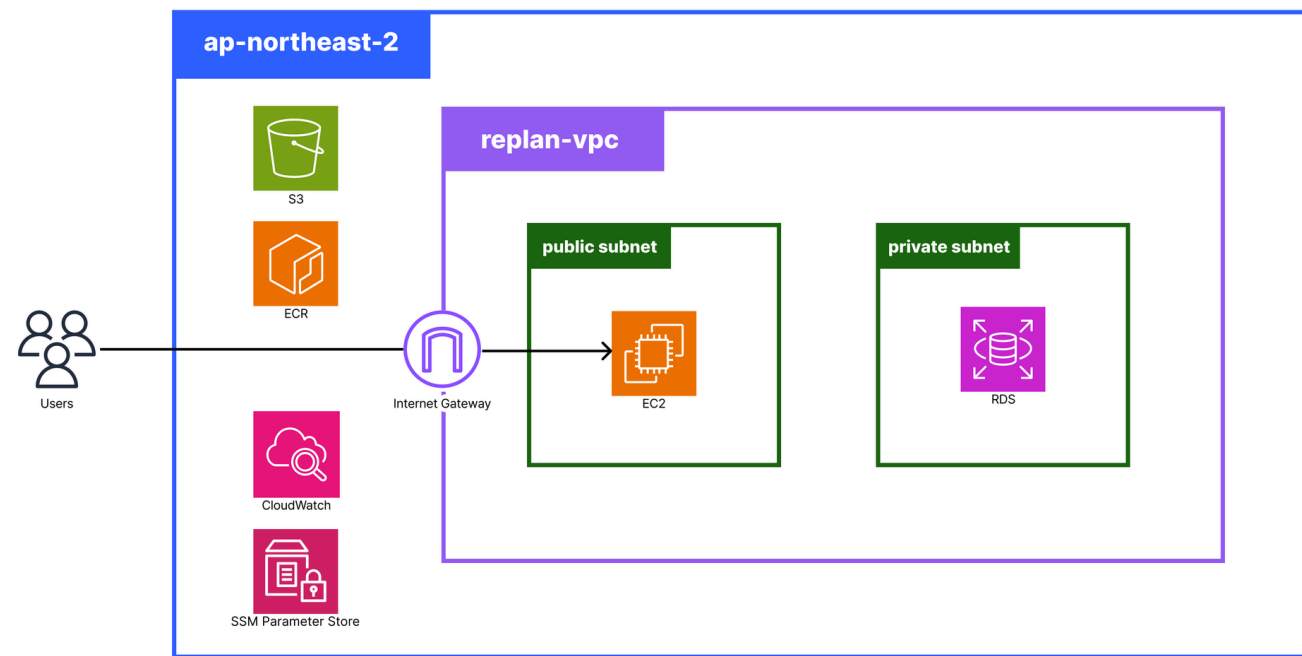
Github Actions

코드 통합부터 배포까지의 전 과정을 자동화하며, OIDC 인증을 활용한 보안 중심의 지속적 통합(CI) 및 배포(CD) 환경 구축

기여

① 인프라 및 CI/CD 파이프라인 설계 & 구현

보안과 자동화를 중심으로 설계한 AWS 클라우드 인프라



[보안 중심 설계]

OIDC 기반 인증으로 고정 액세스 키를 배제하고 **IAM 최소 권한 원칙**을 적용. RDS를 Private Subnet에 배치해 **외부 접근을 차단**하고, SSM Parameter Store로 **환경 변수를 분리**하여 관리.

[컨테이너 기반 배포 자동화]

GitHub Actions로 이미지 빌드부터 ECR 푸시, SSH 기반 앱 갱신까지 **전 과정을 자동화**.

[운영 효율성 및 데이터 관리]

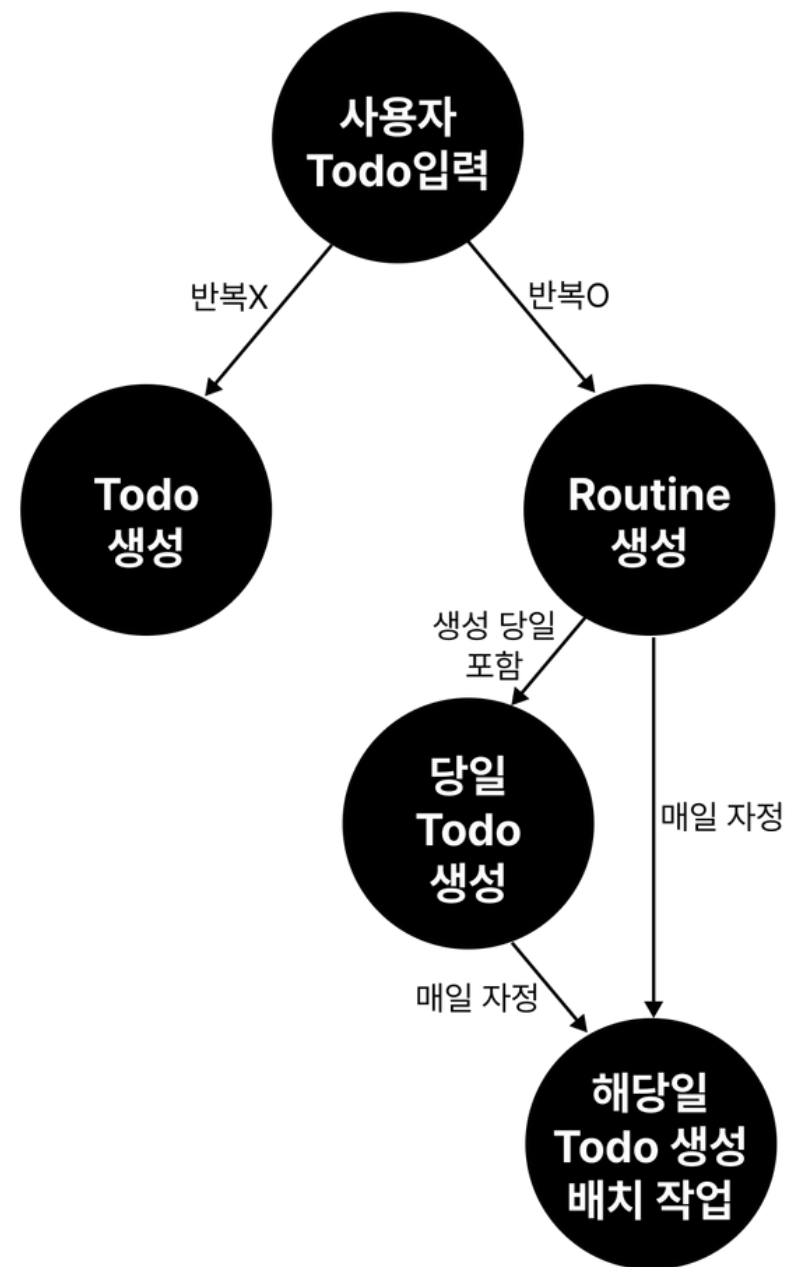
Docker Compose로 3개의 컨테이너를 단일 호스트에서 운영하여 **배포 일관성을 확보**하고, Redis 캐싱과 Nginx 리버스 프록시로 **서비스 응답 성능을 향상**. AWS RDS로 데이터를 독립적으로 운영하고, Flyway로 **스키마 변경 이력을 버전 관리**.

[모니터링 및 관측성]

CloudWatch로 **인프라 리소스와 로그를 실시간 수집**하여 장애 발생 시 신속한 원인 파악 및 대응이 가능한 **관측성(Observability) 확보**

기여

② 반복 Todo 루틴화 및 생성 배치작업



기능 개요

반복 Todo를 관리하기 위한 Routine 시스템을 설계하고 구현했다. 지정한 반복 규칙(일별/주별/월별)에 따라 Todo가 자동 생성된다.

[DB 설계 결정]

반복 Todo의 DB 설계 방식으로 세 가지 접근을 검토했다. 인스턴스를 미리 모두 생성하는 Materialized 방식은 데이터 폭발 문제가 있고, 규칙만 저장하는 Rule-based 방식은 조회 로직이 복잡해진다. 최종적으로 **규칙(Routine)과 인스턴스(Todo)를 분리하는 구조**를 선택했다.

[요일 저장에 Bitmask 적용]

WEEKLY 루틴의 다중 요일 선택을 저장하기 위해 INT 타입 컬럼에 **비트마스크**를 적용했다. DayOfWeekBit Enum을 정의하여 toBitmask() / fromBitmask() 변환 로직을 캡슐화하고, **비트 연산으로 요일 포함 여부를 판별**한다.

[루틴 생성 즉시 오늘치 Todo 생성]

루틴이 생성된 당일치 Todo는 즉시 생성해야 하는 요구사항이 있었다. 배치와 루틴 생성 시점이 동일한 로직을 공유하도록 createTodosForDate(Routine, LocalDate) 메서드로 분리했다. 이를 통해 **중복 코드 없이 두 진입점에서 동일한 규칙을 재사용**한다.

[배치 작업 (매일 자정)]

Spring Scheduler를 활용해 매일 자정(Asia/Seoul 기준)에 전체 루틴을 조회하고, 그 날짜에 해당하는 루틴의 Todo를 **일괄 생성하는 배치**를 구현했다. Clock을 빈으로 등록하여 테스트 시 Clock.fixed()로 교체해 시간 의존 로직을 검증할 수 있도록 했다.

TokenUs

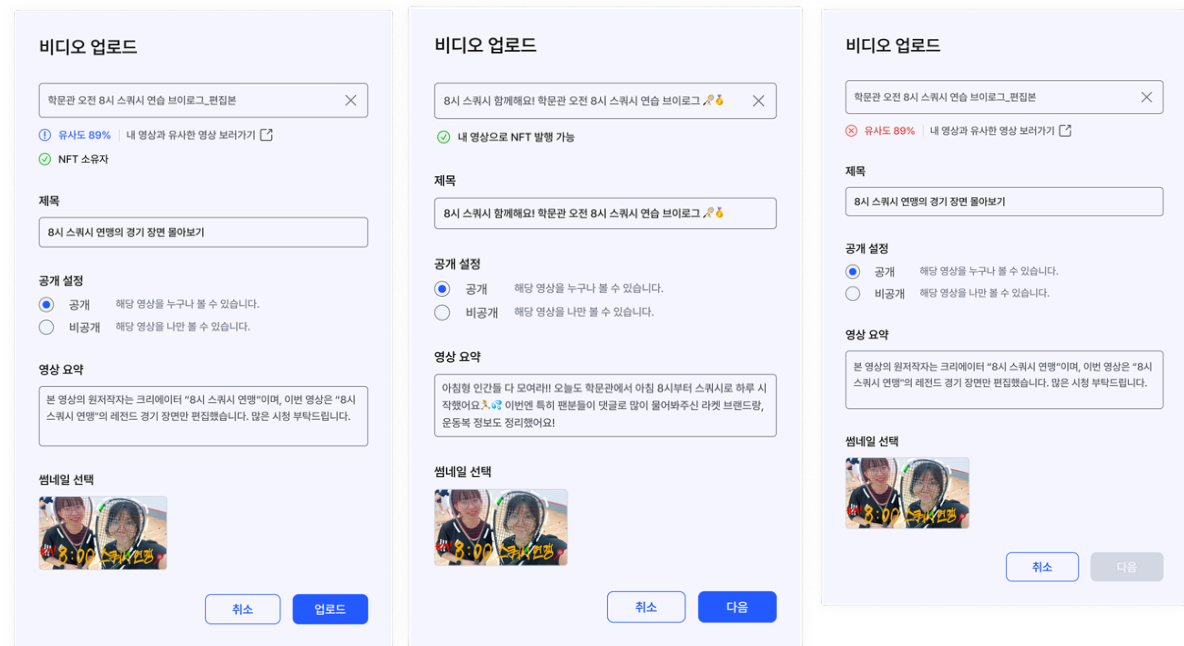


개요

무별한 영상 무단 복제를 방지하고 원저작자의 권리를 보호하기 위한 블록체인 기반 영상 공유 플랫폼, TokenUs

핵심 기능

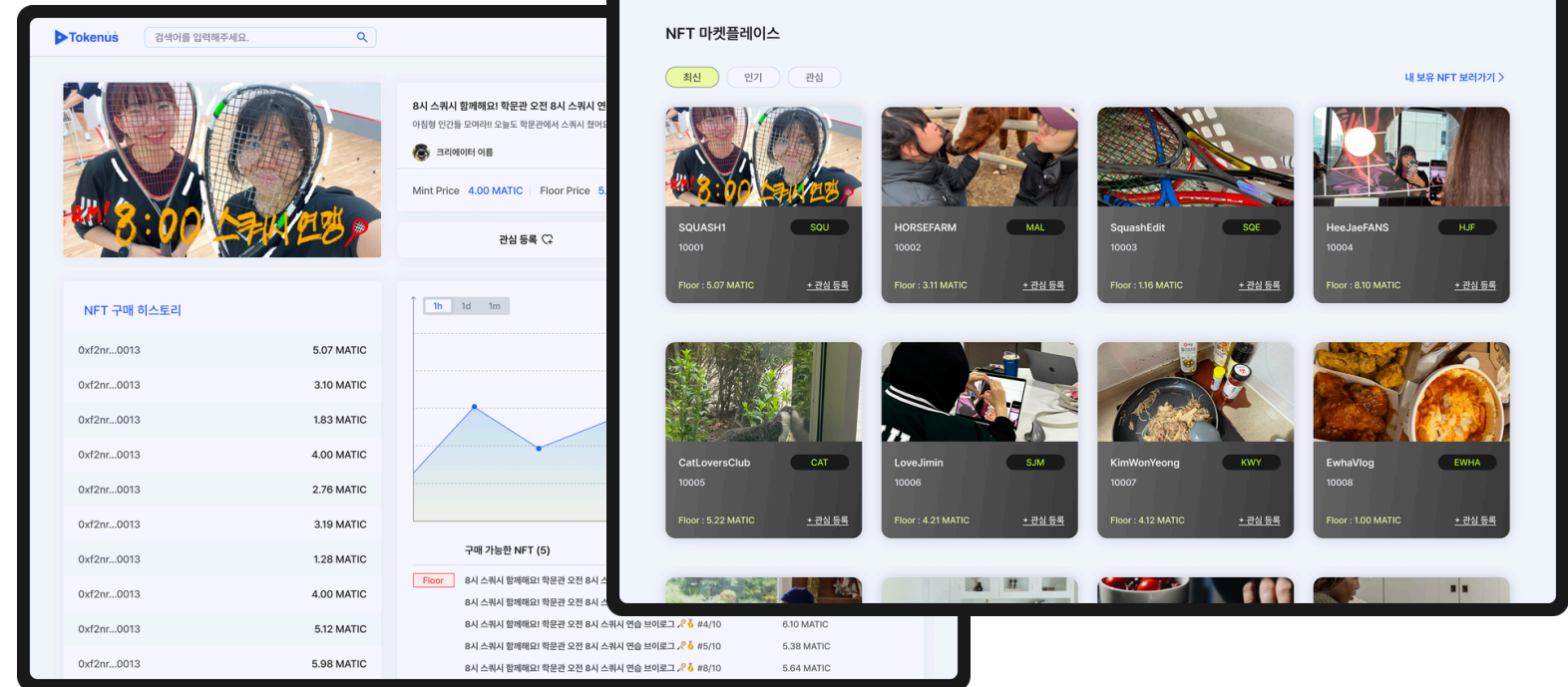
영상 유사도 검사



- 업로드하는 영상에 대해 자동으로 다른 영상들과의 **유사도 검사**를 진행.
- 영상의 **고유성을 보장함**으로써 **NFT 발행의 정당성**을 기술적으로 확보.

영상 NFT 거래

영상 NFT 발행



- 영상 콘텐츠를 **NFT로 발행**
- NFT의 발행부터 소유권 이전 및 거래 이력 관리까지 **전 과정을 자동화**

기술스택



Spring Boot

강력한 생태계와 안정성을 바탕으로 의존성 주입과 JPA를 통해 비즈니스 로직에만 집중할 수 있는 생산성 확보를 위해 선택



EC2

서비스의 규모에 따라 유연한 확장이 가능한 가상 서버를 구축하기 위해 활용

ECR

Docker 이미지를 안전하고 프라이빗하게 관리하며, EC2로의 배포 파이프라인 효율을 높이기 위해 사용

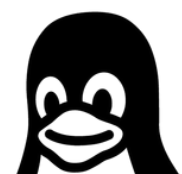
S3

정적 파일(이미지, 영상 등)의 높은 내구성과 가용성을 보장하는 객체 스토리지 서비스로 활용



Docker

개발-테스트-운영 환경의 환경 일관성을 유지하고, 애플리케이션의 독립적인 실행 환경 격리를 위해 사용

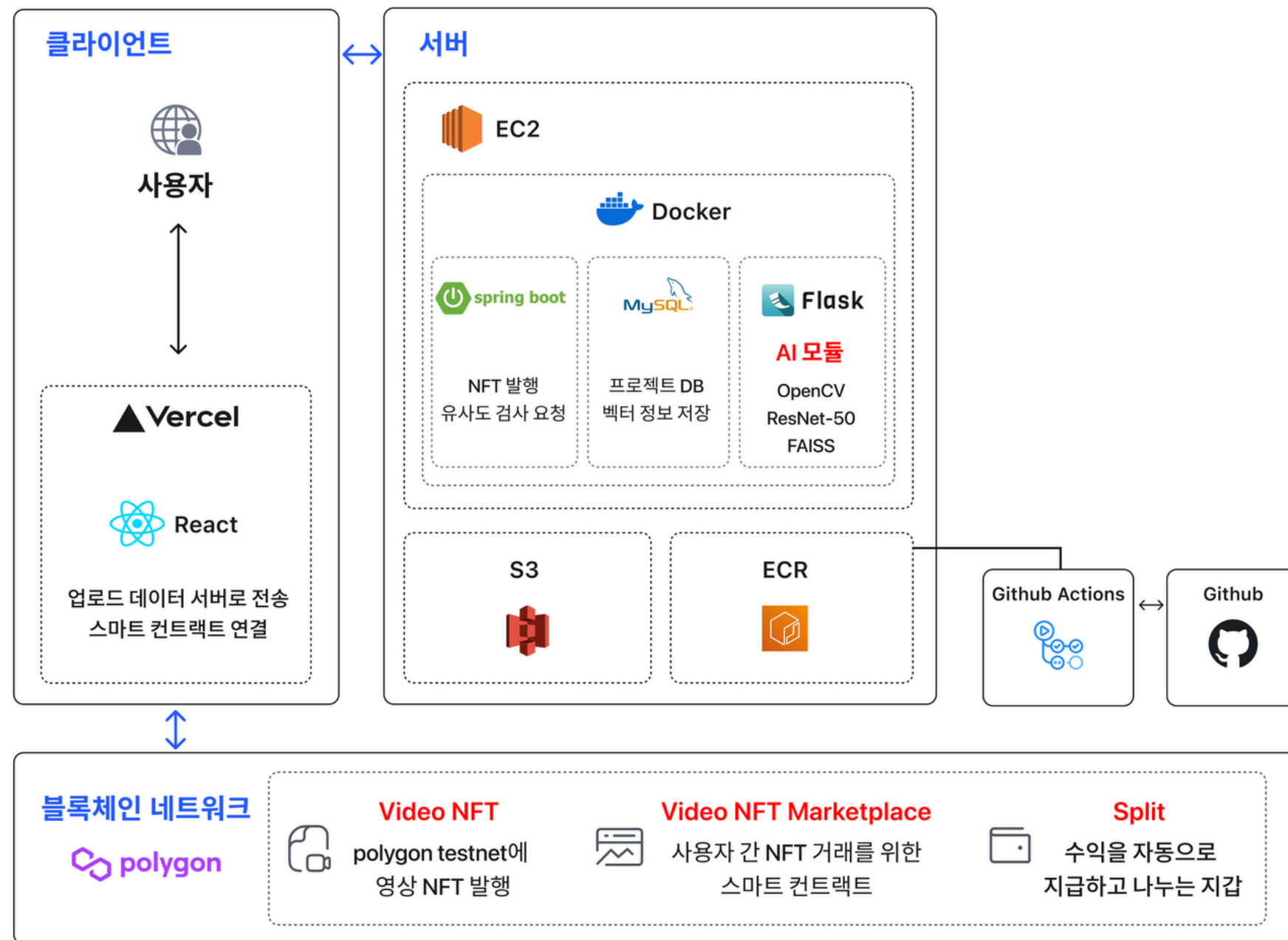


Linux
(Ubuntu)

서버 환경의 표준이자 오픈 소스 생태계의 풍부한 패키지를 활용할 수 있으며, 시스템 리소스 효율성을 극대화하기 위해 서버 OS로 채택

기여

① 아키텍처 설계



제한된 리소스 내에서

백엔드, DB, 머신러닝, 블록체인 서버 사이의 연결을 원활히 하고

효율적인 운영을 위한 아키텍처 설계

[데이터 안정성 확보]

모든 멀티미디어 데이터(영상, 이미지)를 서버 로컬이 아닌 AWS S3에 저장함으로써, 서버 장애 시에도 **데이터 유실을 방지**

[환경 격차 해소]

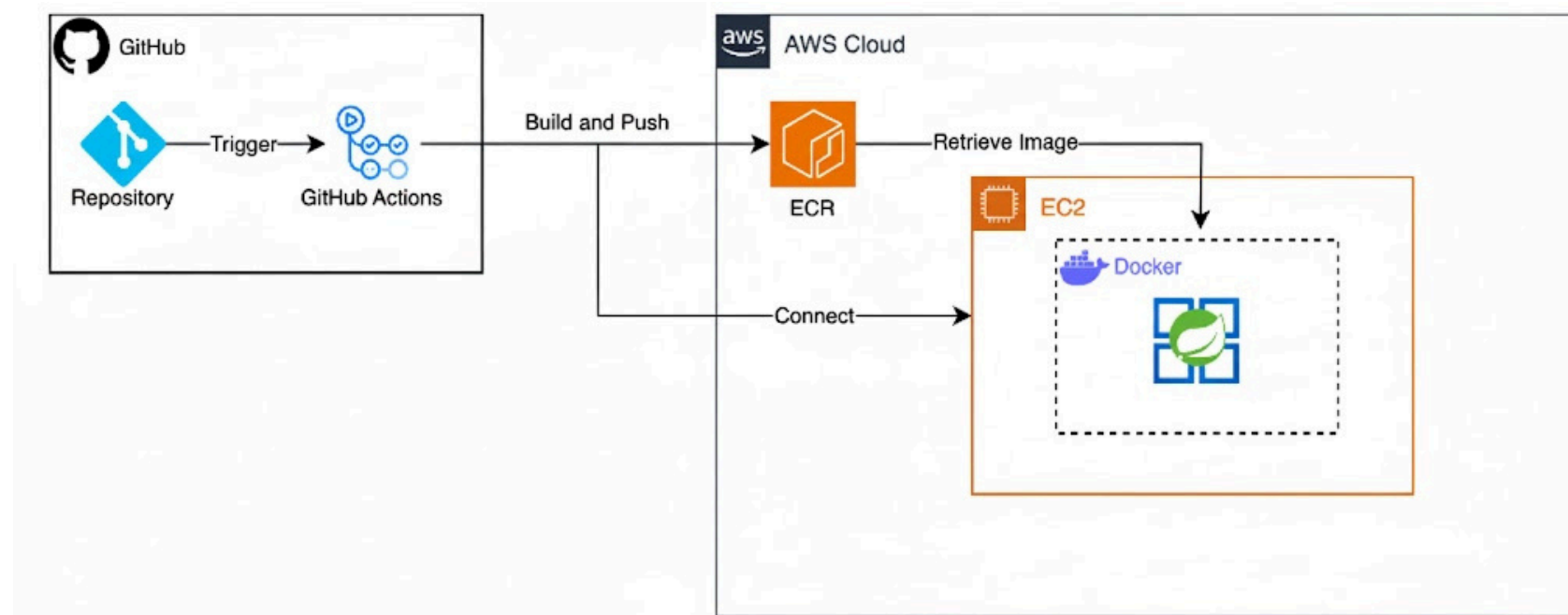
Docker를 활용하여 로컬 개발 환경과 EC2 운영 환경의 **일치성을 확보**하고, 배포 시 발생할 수 있는 **환경 문제를 최소화**

[보안 및 관리]

AWS ECR을 사용하여 프라이빗 컨테이너 레지스트리를 구축함으로써 **이미지 보안을 강화**하고, 외부 노출 없이 안전하게 이미지를 관리

기여

② CI/CD 파이프라인 구축



[컨테이너 기반 자동화 프로세스 구축]

- 애플리케이션 **컨테이너화**로 환경 **일관성 보장** 및 **ECR**을 통한 이미지 **보안 강화**
- 빌드부터 서버 갱신까지 **전 과정 자동화**로 배포 리드타임 단축 및 에러 방지
- SSH Key 및 AWS 자격 증명을 GitHub Secrets로 관리하여 **민감 정보 유출 방지**

[효율적인 운영을 위한 배포 아키텍처]

- 최신 이미지 Pull 및 컨테이너 재실행 방식의 **CD 파이프라인 최적화**
- 빌드와 실행 단계 분리로 **저사양(Free Tier)** 환경 내 인프라 부하 최소화

③ API 설계 및 구현

[코어 비즈니스 API]

사용자 경험을 고려한 회원가입, 영상 업로드 및 조회 등 서비스 전반의 **RESTful API 설계**

[인프라 및 외부 연동]

- AWS S3 연동을 통한 안정적인 **멀티미디어 데이터 핸들링**
- **SmartContract 통신 로직 구현**으로 블록체인 기반 데이터 보안성 강화
- **ML 서버 API 연동**을 통해 데이터 분석 결과의 실시간 서비스 반영 프로세스 구축

트러블 슈팅

① WebSocket 도입 및 비동기 처리를 통한 ML 연산 타임아웃 문제 해결

문제 상황

```
유사도 결과 수신:
Object {
  elapsed_time: 48.51
  max_segment_similarity: 0.9915997445583343
  message: "Similarity check failed"
  passed: false
  similar_video_id: null
  similar_video_url: null
  token_ids: null
  video_uri: "https://woobaby.s3.ap-northeast-2.amazonaws.com/..."
}
```

[현상]

ML 모델의 연산 복잡도와 영상 파일 크기에 따라 처리 시간이 API 서버의 최대 대기 시간(**Timeout**)을 초과하게 됨.
결과적으로 사용자는 **결과를 받지 못한 채 연결이 끊기**는 현상이 발생.

원인 분석

[ML 연산의 병목]

영상 데이터의 특성상 실시간 응답을 보장하기 어려운 무거운 작업임에도 불구하고, HTTP 요청-응답 구조인 **동기 방식**으로 처리하려 한 것이 근본적인 원인

[자원 낭비]

클라이언트가 응답을 받기 위해 무한정 연결을 유지해야 하므로 **서버 자원 효율성**이 떨어지는 구조

해결

WebSocket 도입

HTTP의 일회성 연결 대신, 클라이언트와 서버 간의 지속적인 연결 통로를 구축.

프로세스 분리

요청 즉시 "접수 완료"를 반환하여 타임아웃 방지.

실시간 결과 Push

ML 연산 완료 시점에 서버가 소켓을 통해 클라이언트에게 결과를 즉시 전송

인사이트

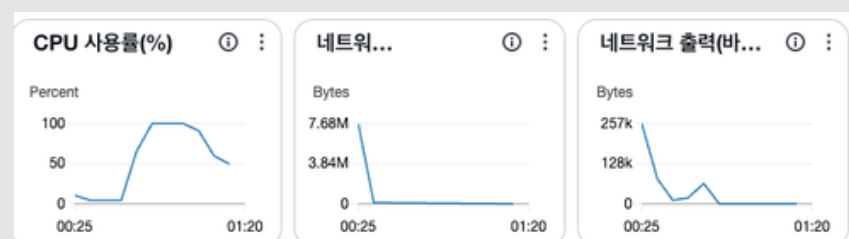
데이터 처리 시 **비동기 방식**과 **지속적 연결의 중요성**을 체감했으며, 서버 자원 효율성을 고려한 아키텍처 설계의 필요성을 배움.

트러블 슈팅

② EC2 Docker 컨테이너 실행 시 CPU 100% 점유 및 서버 불능 문제 해결

문제 상황

```
ssh: connect to host ec2-13-125-242-186.ap-northeast-2.compute.amazonaws.com port 22: Operation timed out
```



[현상]

테스트 과정에서 CI/CD 파이프라인을 통해 EC2에 컨테이너를 실행했으나, 직후 **SSH 접근이 불가능**해지는 현상 발생.

간신히 접근에 성공하더라도 EC2 인스턴스가 정상 작동하지 않았으며, 모니터링 결과 **CPU 사용률이 100%를 상회**함

원인 분석

[초기 구동 부하]

Spring Boot 애플리케이션이 배포 초기 단계에서 컨테이너 실행 및 의존성 로딩 과정에서 CPU 사용률이 급격히 증가함.

[환경 제약]

프리티어 EC2 인스턴스의 **제한된 자원** (RAM 1GB, 낮은 CPU 성능)으로 인해 **초기 구동 시 발생하는 부하를 감당하지 못하고** CPU 사용률이 100%에 도달하며 시스템이 불안정해짐.

해결

애플리케이션 및 배포 설정 최적화

- **JPA 설정 변경:** ddl-auto를 none으로 설정하여 서버 기동 부하 감소
- **리소스 제한:** 컨테이너 CPU(0.5) 및 메모리(512M) 제한을 통해 시스템 다운 방지

인스턴스 메모리 환경 개선

부족한 RAM을 보완하기 위해 HDD/SSD의 일부를 메모리처럼 사용하는 **스왑 메모리 (2GB)를 설정**함.

인사이트

제한된 리소스를 가진 클라우드 환경에서는 애플리케이션의 **설정 최적화**뿐만 아니라, **시스템 레벨의 리소스 관리**가 필수적임을 학습

SASM

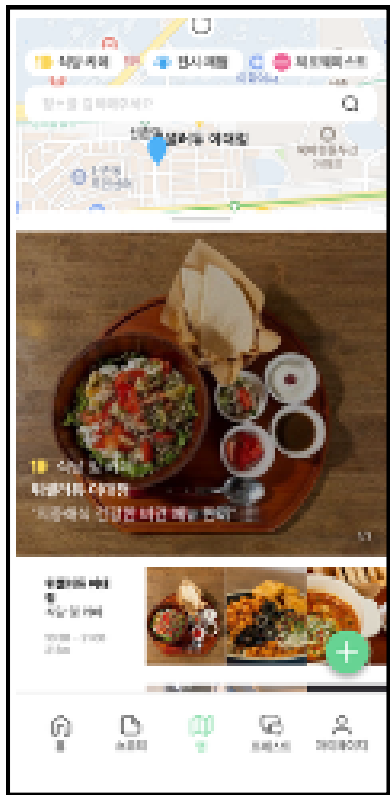


개요

SDGs 목표 실현에 기여하는 여가 공간을 ‘지속가능한 공간’으로 정의하고, 이에 대한 지도화 및 큐레이팅 서비스를 제공

핵심 기능

Place(Map)



SDGs 17개의 목표 실현에 기여하는 공간을 지속 가능한 공간으로 정의하고, 공간을 모아 지도화

Forest



커뮤니티의 일종으로, 유저가 자유롭게 본인의 생각을 공유

Story



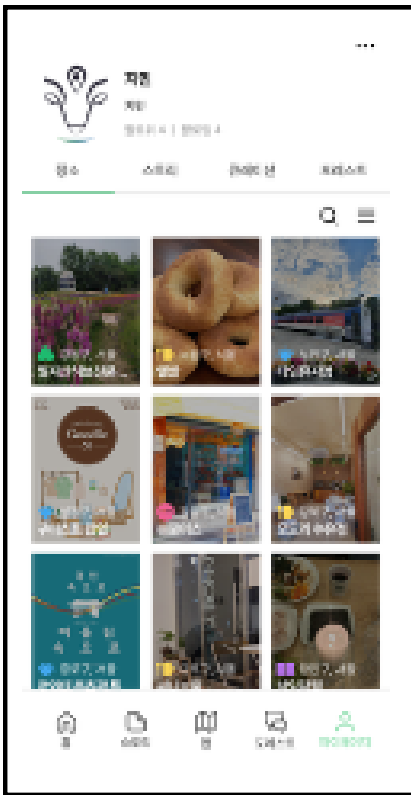
특정 Place에 대한 이야기

Curation(Home)



관련 Story를 엮어 유저들에게 제공

MyPage



팔로잉 기능, 좋아요를 누르거나 리뷰를 작성한 내용에 대한 아카이빙 제공

기술스택

Core Development



Django

기존 프로젝트의 Service Layer 패턴을 분석하여, 신규 기능을 추가할 때 기존 비즈니스 로직과 충돌하지 않도록 일관성 있는 코드를 작성함



MySQL

수백 건의 데이터를 다루는 테이블에 신규 기능을 추가하며, 서비스 성능 저하를 막기 위해 실행 계획을 확인하고 기존 인덱스를 타는지 체크하며 쿼리를 최적화함

Infrastructure & Environment



EC2

RDS

S3

EC2와 RDS로 구성된 AWS 클라우드 인프라 구조를 파악하고, 배포 프로세스에 참여하며 백엔드 서비스가 실제 유저에게 전달되는 전체 흐름을 익힘



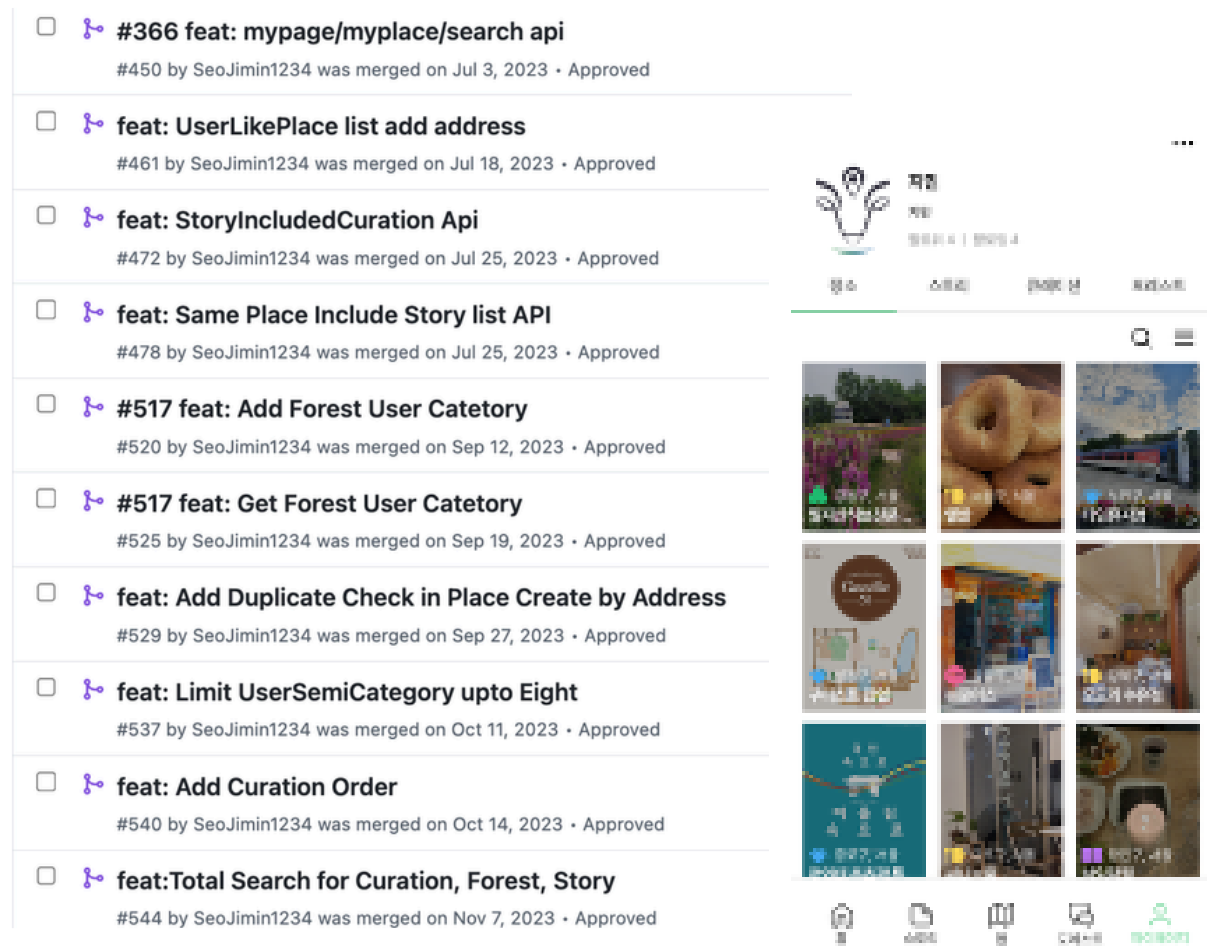
Docker

Docker 기반의 컨테이너 환경을 이해하고, 로컬 개발 환경과 실제 운영 서버 간의 환경 격차를 최소화하여 개발 및 테스트를 진행

기여

① 신규 기능 설계 및 구현

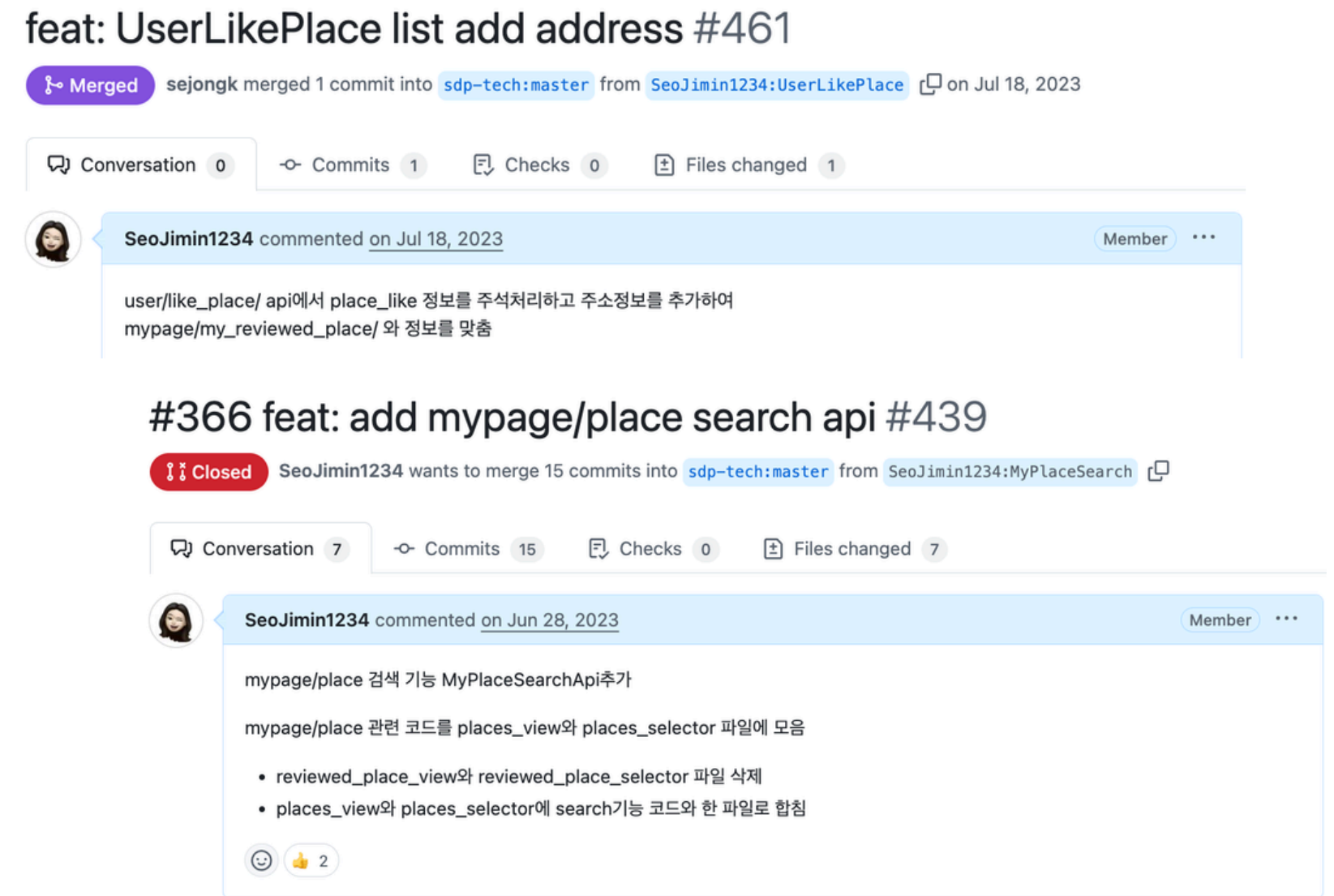
기존 시스템과 정합성을 유지하는 MyPage 서비스 설계 및 구현



- 기존의 User, Place, Story 데이터 모델을 분석하여 **MyPage 신규 API 설계**.
- 기존 기능과의 **코드 충돌을 방지**하기 위해 서비스 레이어 분리 및 인터페이스 활용..
- 필요 정보를 효율적으로 조회할 수 있도록 **최적화된 API** 제공.

② 코드 가독성 및 유지보수성

확장성을 고려한 레거시 코드 개선 및 API 개발



- 프로젝트 **공통 컨벤션 및 디자인 패턴**을 준수하여 기존 코드와의 통일성 유지.

기여

③ 운영 이슈 대응 및 버그 수정



📌 에러 목록

• 에러 작성 시 최대한 구체적으로 작성 부탁드립니다!
(사진 캡처, 발생 위치와 구체적인 현상 작성)

📋 표

↑ 우선순위 ▾

📌 해결: 체크 표시됨 ▾ + 필터

☰ 우선순위

☰ 테크 메모

☰ 발생 위치

☑ 해결

Aa 이름

1순위		MyPage	👍	📄 유저 프로필 수정
		Map		
1순위		Map	👍	📄 맵에서 큰 화면 볼 때 맞춰서 적당히
1순위	BE 구현 완료(#544)	etc	👍	📄 통합 검색
1순위		Map	👍	📄 리뷰에 사진이 없
1순위		Story	👍	📄 스토리 목록형식(
1순위		MyPage	👍	📄 유저 정보 수정, <
2순위		Forest	👍	📄 BoardDetail→ 튼을 누르면 정상 X 오류 ⇒ usecc
2순위		Map	👍	📄 홈에서 장소추천 당하는 목록들이 목록들이 뜸)

☐  #472 fix: Add SamePlaceStory informations

#481 by SeoJimin1234 was merged on Jul 30, 2023 • Approved

☐  fix: Remove duplication at mypick_forest

#487 by SeoJimin1234 was merged on Aug 1, 2023 • Approved

☐  #487 fix: Remove my_forest Duplication

#488 by SeoJimin1234 was merged on Aug 2, 2023 • Approved

☐  fix: Add Nickname in Mypage/Forest

#496 by SeoJimin1234 was merged on Aug 9, 2023 • Approved

☐  fix: Modify Story vegan_category Selector

#501 by SeoJimin1234 was merged on Aug 14, 2023 • Approved

☐  fix: Return Nickname at Curation List

#506 by SeoJimin1234 was merged on Aug 18, 2023 • Approved

☐  fix: Replace preview with Summary in Story Search

#509 by SeoJimin1234 was merged on Aug 20, 2023 • Approved

☐  #520 fix: User semi Cateory Blank True

#526 by SeoJimin1234 was merged on Sep 23, 2023 • Approved

☐  #540 fix: Curation Order set default

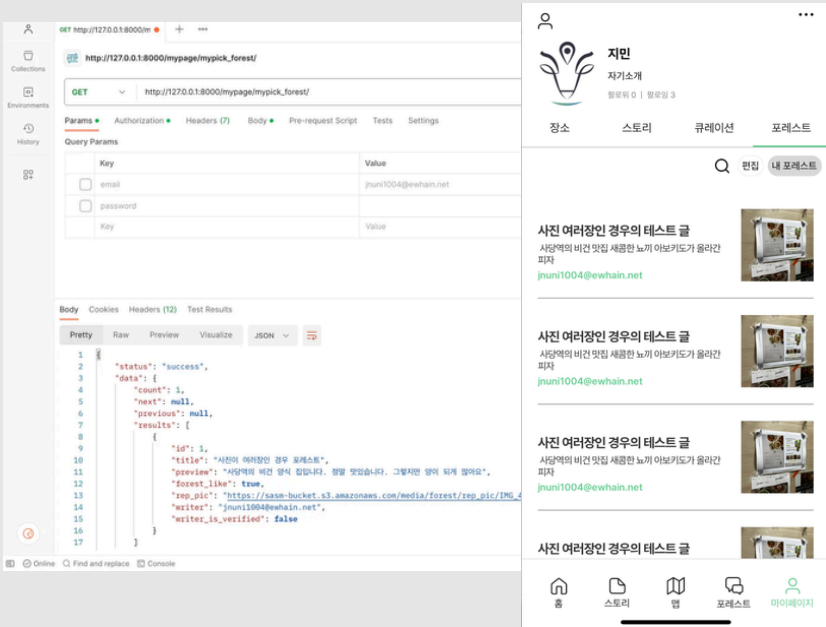
#541 by SeoJimin1234 was merged on Oct 14, 2023 • Approved

- 출시 전 자체적인 **탐색적 테스트**를 수행하여 서비스 플로우 상의 결함을 사전에 발견하고 수정.
- 사용자 피드백 및 QA 과정에서 접수된 엣지 케이스 **버그**를 **파악**하고, 로직 보완을 통해 **서비스 안정성 향상**.
- 서버 로그 분석**을 통해 운영 환경에서 발생하는 런타임 에러를 실시간으로 추적 및 해결.

트러블 슈팅

ManyToMany 관계 필터링 시 객체 중복 리턴 문제 해결

문제 상황



[현상]

마이페이지의 '내가 찜한 포레스트' 리스트 조회 시, 특정 게시글에 포함된 **해시태그 개수만큼 동일한 객체가 중복되어 리턴되는** 현상 발생.

데이터베이스에 객체가 여러 개 생성된 것은 아니나, 조회(GET) 결과에서만 동일한 ID의 데이터가 반복됨.

원인분석

[Q 객체를 이용한 복합 필터링]

search 매개변수를 통해 제목, 부제목, 내용 및 해시태그 이름을 검색 조건으로 설정.

[SQL Join의 특성]

ManyToMany 관계인 해시태그 필드를 필터링 조건에 포함하면서 SQL 내부적으로 **JOIN**이 발생함. 이때 하나의 포레스트 객체가 여러 개의 해시태그를 가질 경우, 데이터베이스 수준에서 **해시태그 개수만큼 행이 생성**되어 Django QuerySet에 그대로 반영됨.

해결

distinct()와 prefetch_related()의 조합

prefetch_related('hashtags')를 사용하여 해시태그 데이터를 효율적으로 가져와 N+1 쿼리 문제를 방지함.

QuerySet 마지막에 **.distinct()**를 호출하여 SQL 레벨에서 중복된 객체를 제거하도록 명시함.

인사이트

Django의 ORM과 ManyToMany 필터링 시 발생하는 **SQL Join의 동작 원리**를 깊이 있게 이해함.